Bharatiya Vidya Bhavan's
SARDAR PATEL INSTITUTE OF TECHNOLOGY
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri (W), Mumbai – 400 058.
Department of Computer Engineering

Lab Assignment

Aim: Comparative Analysis of Probabilistic Primality Testing Algorithms

1. Problem Statement

To implement and experimentally compare the performance of the following probabilistic primality testing algorithms:

- Fermat Primality Test
- Euler Primality Test
- Miller–Rabin Primality Test

using a common dataset and evaluating their effectiveness in identifying prime and composite numbers.

2. Learning Outcomes

After completing this experiment, students will be able to:

- Implement three probabilistic primality testing algorithms.
- Understand the difference between deterministic and probabilistic primality tests.
- Compare the accuracy of different primality tests.
- Measure and compare execution time.
- Analyze false positive cases, particularly Carmichael numbers.
- Explain why Miller–Rabin is widely used in practical cryptographic systems such as RSA.

3. Software Requirements

- Python 3.x
- Pandas (optional)
- Time module (for execution time measurement)

4. Dataset

Each student will be provided with the file:
primality_dataset_1000.csv
The dataset contains:

Category	**Count**
Prime Numbers	250
Composite Numbers	700
Carmichael Numbers	50
Total	**1000**

Each row contains the actual classification of the number, which must **not** be used during testing. The **Actual_Type** column is to be used only for evaluation after all tests have been completed.

5. UID-Based Test Base Selection

To ensure that each student performs a unique experiment while using the same dataset, derive the test bases from your **10-digit UID**.

Suppose the UID is ***2022300019***

Generate the following bases:

Base 1 = (Last two digits [19] % 97) + 2
Base 2 = (Sum of digits [14] % 97) + 2
Base 3 = ((First four digits [2022] + Last four digits [0019]) % 97) + 2
Base 4 = (sum of digits at odd positions [2+2+3+0+1] % 97) + 2
Base 5 = (sum of digits at even positions [0+2+0+0+9] % 97) + 2

Each number must be checked against 5 bases. If the selected base is not valid for a given number (e.g., base ≥ n or gcd(base, n) ≠ 1), generate a new valid base randomly and record it in your report.

6. Experimental Procedure

Step 1: Load the provided CSV dataset.

Step 2: For every number in the dataset:

- Perform the Fermat Primality Test.
- Record the result as **Prime** or **Composite**.
- Measure the execution time.

Populate the columns:

- Fermat_Result
- Fermat_Time_ms

Step 3: Repeat the process using the Euler Primality Test.

Populate:

- Euler_Result
- Euler_Time_ms

Step 4: Repeat the process using the Miller–Rabin Primality Test.

Populate:

- MillerRabin_Result
- MillerRabin_Time_ms

Step 5: Compare the obtained results with the **Actual_Type** column and compute the following performance metrics for each algorithm.

- Correct Classifications
- Incorrect Classifications
- False Positives
- False Negatives
- Accuracy
- Average Execution Time

7. Result Table

Detailed Table

Sr. No.	Number	Actual Type	Fermat	Euler	Miller Rabin	Execution Time Fermat (ms)	Execution Time Euler (ms)	Execution Time MR (ms)
1	561	Carmichael						
2	678931	Composite						
3	138211	Prime						
...	...	...						

Summary Table

Algorithm	Correct	Incorrect	False Positives	False Negatives	Accuracy (%)	Avg. Time (ms)
Fermat						
Euler						
Miller-Rabin						

8. Observations

While analyzing the complete dataset, answer the following:

1. Which algorithm produced the highest accuracy?
2. Which algorithm required the least execution time?
3. How many Carmichael numbers were incorrectly identified by Fermat's test?
4. Did Euler's test improve upon Fermat's results?
5. Why is Miller–Rabin preferred for practical cryptographic applications?
6. How does the choice of base influence the outcome of the probabilistic tests?

Deliverables

Submit the following:

- Python source code for all three algorithms.
- Completed primality_dataset_1000.csv.
- Experimental analysis tables.
- **Graphs showing:**
  - Accuracy of each algorithm.
  - Average execution time.
  - Number of false positives.
- A one-page conclusion discussing your observations.